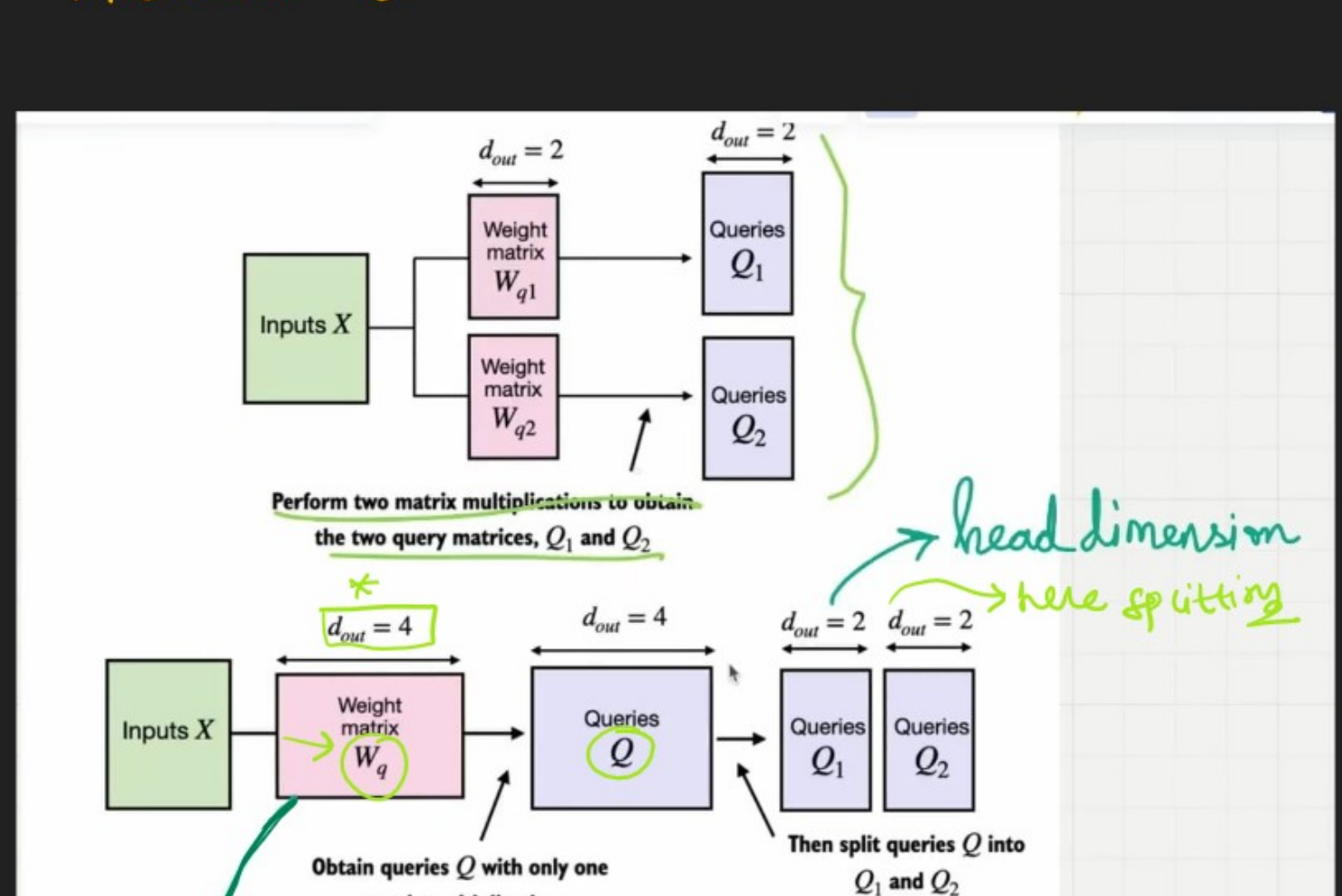
Lecture-18 (Multi-Head Attention)

with weight splits

(B.) Implementing multi-head attention with weight splits

① Instead of maintaining 2 seperate classes: multi head attention wrapper and casual attention, we combine both of these into a single multiple Attention class.



$d_{out} = 2$
Weight matrix W_{q1}
Weight matrix W_{q2}
Inputs X
Queries Q_1
Queries Q_2
Perform two matrix multiplications to obtain the two query matrices, Q_1 and Q_2
$d_{out} = 4$
$d_{out} = 2$
Weight matrix W_q
Queries Q
Queries Q_1
Queries Q_2
Obtain queries Q with only one matrix multiplication
Then split queries Q into Q_1 and Q_2

→ head dimension
→ here spliting

Number of attention heads is specified:

Head dimension = $\frac{d\_out}{n\_head}$

* Let's us take a simple example

Step-1: Start with the input

$(b, num\_token, \underbrace{d\_in}_{\text{dimension of input token}}) = (1, 3, 6)$

$X = \text{torch.tensor}([[[1.0, 2.0, 3.0, 4.0, 5.0, 6.0],$
$[1.0, 2.0, 3.0, 4.0, 5.0, 6.0],$
$[1.0, 2.0, 3.0, 4.0, 5.0, 6.0]]])$ → 3 × d out

batch = 1
num-token = 3, d-in = 6

Step-2: Decide d_out, num_heads

d_out = 6, num_heads = 2

head_dim = $\frac{6}{2} = 3$

Step-3: Initialize trainable weight matrices for key, Query, value (W_k, W_q, W_v)

6×6 → din × dout: $[W_q]$ 6×6: $[W_k]$ 6×6: $[W_v]$

Step-4: Calculate keys, Queries, value matrices

(Input * W_k Input * W_q Input * W_v)

[Keys] [Queries] [Values]

Keys (1×3×6) Queries (1×3×6) Values (1×3×6)

1 → batch-size, 3 → num-token, 6 → d-out

Step-5: Unroll last dimension of keys, Queries and values to include num-heads and head dim.

head_dim = d_out/num_heads = 6/2 = 3

(b, num_tokens, d_out) → (b, num_token, num_heads, head_dim)

(1, 3, 6) → (1, 3, 2, 3)

Reshaped Queries matrix → (1, 3, 2, 3) (No. of heads; No. of token)

```
tensor([[[—,—,—],
         [—,—,—]],
        [[—,—,—]
         [—,—,—]],
        [[—,—,—],
         [—,—,—]]])
```

Reshaped Keys matrix

```
tensor([[[—,—,—],
         [—,—,—]],
        [[—,—,—]
         [—,—,—]],
        [[—,—,—],
         [—,—,—]]])
```

Reshaped Values matrix

```
tensor([[[—,—,—],
         [—,—,—]],
        [[—,—,—]
         [—,—,—]],
        [[—,—,—],
         [—,—,—]]])
```

Step-6: Group matrices by "no. of heads"

(b, num_token, num_head, head_dim) → (b, num_head, num_token, head_dim)

(1, 3, 2, 3) → (1, 2, 3, 3)

Transposed Queries matrix → shape (1, 2, 3, 3)

```
tensor([[[— — —],
         [— — —],      } Head-1
         [— — —]],

        [[— — —],
         [— — —],      } Head-2
         [— — —]]])
```

Transposed keys matrix

```
tensor([[[— — —],
         [— — —],      } Head-1
         [— — —]],

        [[— — —],
         [— — —],      } Head-2
         [— — —]]])
```

Transposed Values matrix

```
tensor([[[— — —],
         [— — —],      } Head-1
         [— — —]],

        [[— — —],
         [— — —],      } Head-2
         [— — —]]])
```

Step-7: Find attention score

Queries * key. Transpose (2,3)

Queries matrix:
```
tensor([[[- - -],
Head-1   [- - -],
         [- - -]],
Head-2  {[- - -],
         [- - -],
         [- - -]],
```
(1, 2, 3, 3)
(b, num_heads, num_tokens, head-dim)

× (multiply)

key. transpose (2,3) =
```
tensor([[[- - -],
Head-1   [- - -],
         [- - -]],
Head-2  {[- - -],
         [- - -],
         [- - -]],
```
(1, 2, 3, 3)
(b, num_heads, head-dim, num_tokens)

Result after multiplication

Queries. key. transpose (2,3) =
```
tensor([[[ - - - ],   the cat sleep
the      [- - -],     Head-1
cat      [- - -],
sleep
         [- - -],
         [- - -],     Head-2
         [- - -],
```
Attention Score

(1, 2, 3, 3)
(b, num_heads, num_tokens, num_tokens)

Step-8: Find the attention weight

→ mask attention scores to implement casual attention.

```
tensor([[[— -inf -inf],
         [— — -inf],
         [— — —],

         [— -inf -inf],
         [— — -inf],
         [— — —],
```

Replace -∞ using softmax

Divide by $\sqrt{head\_dim} = \sqrt{d\_out/num\_heads}$

$\Rightarrow \sqrt{6/2} = \sqrt{3}$

```
tensor([[[— -inf -inf],
         [— — -inf],
         [— — —],

         [— -inf -inf],
         [— — -inf],
         [— — —],
```

Apply softmax to tensor

```
tensor([[[— — —], → sum up to 1
         [— — —], → sum up to 1
         [— — —], → sum up to 1

         [— — —], → sum up to 1
         [— — —], → sum up to 1
         [— — —], → sum up to 1
```

we can also implement dropout after this.

Attention weight
(1, 2, 3, 3)
(b, num-heads, num-token, num-token)

context vector = Attention weight * values.

(b, num_heads, num-token, num. token) (b, num_heads, num-token, head-dim)

[attention scor] * [Values] ⇒ [context vector output]

(1, 2, 3, 3) (1, 2, 3, 3) (1, 2, 3, 3)

(b, num_heads, num-token, head-dim)

Step-10: Refactor context vector

(b, num_heads, num-token, head-dim) → (b, num-token, num-heads, head-dim)

```
tensor([[[- - -],
         [- - -],

         [- - -],
         [- - -],

         [- - -],
         [- - -]]])
```
→ (1, 3, 2, 3)
(b, num-token, num-heads, head-dim)

Step-11: combine heads

```
tensor([[[- - -],
         [- - -],   } token 1

         [- - -],
         [- - -],   } token 2

         [- - -],
         [- - -]]]) } token 3
```

Flatten each token output into each row

```
tensor([[[ - - - - - - ], → the
         [ - - - - - - ], → cat
         [ - - - - - - ]]]) → sleep
```

(b, num-tokens, d_out)
(1, 3, 6)

→ final context vector

Here in final, we can not see no. of heads.